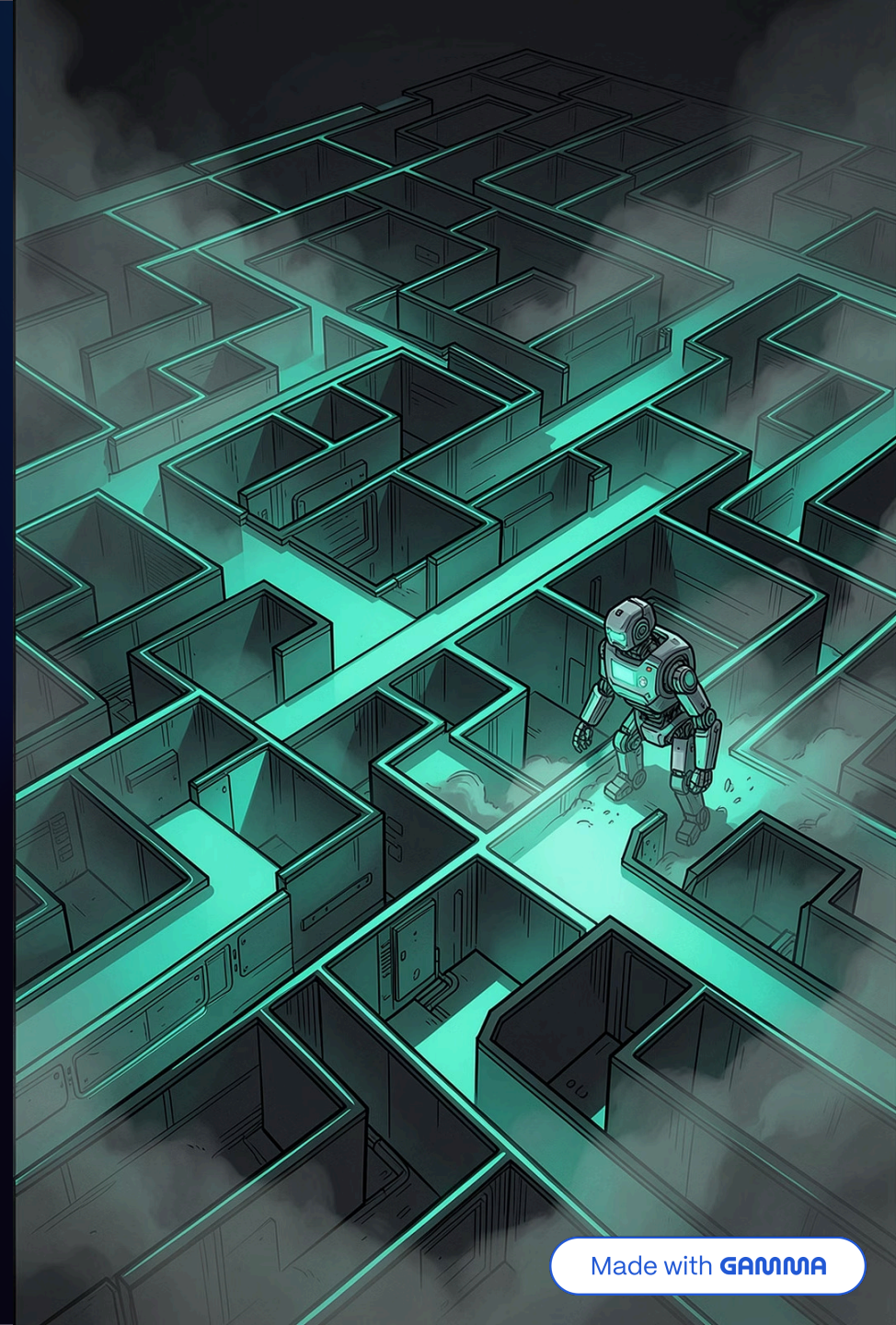


Imitational Learning: FSC vs Behavioral Cloning

by Martinelli Davide

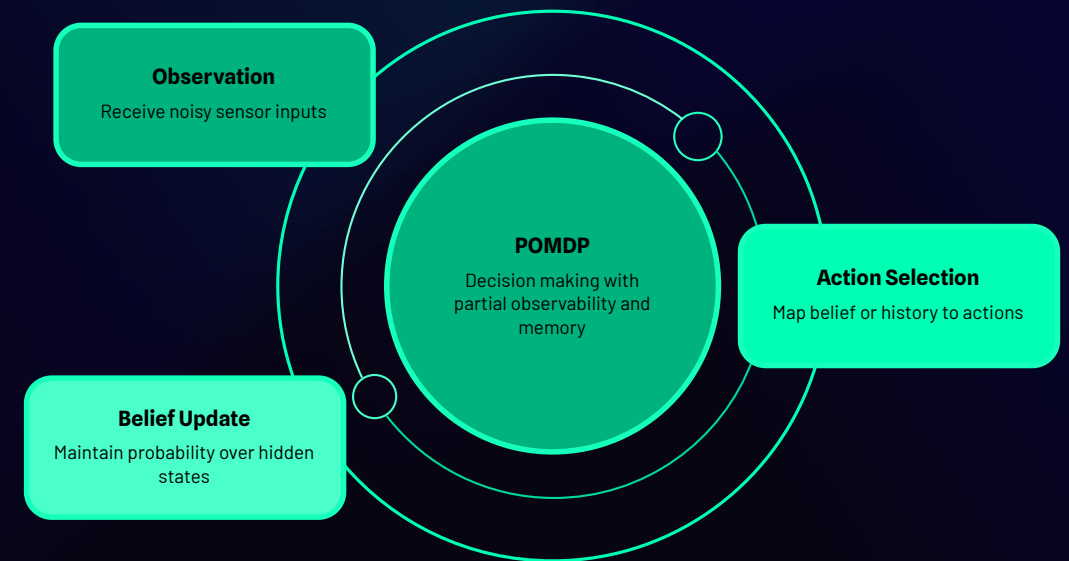
Comparing **Finite State Controllers** and **LSTM-based Behavioral Cloning** in a partially observable gridworld — exploring the trade-off between interpretability and performance.



What is a POMDP?

A **partially observable Markov decision process** (**POMDP**) is a generalization of a **Markov decision process** (MDP). It is assumed that the **system dynamics** are determined by an MDP, but the agent cannot directly observe the underlying state.

Unlike an MDP, where the optimal policy simply maps the current state to an action, **solving a POMDP typically requires memory: the optimal policy is a mapping from the history of observations** (often formalized as a **belief state** or compressed into an internal memory) **to the actions**.



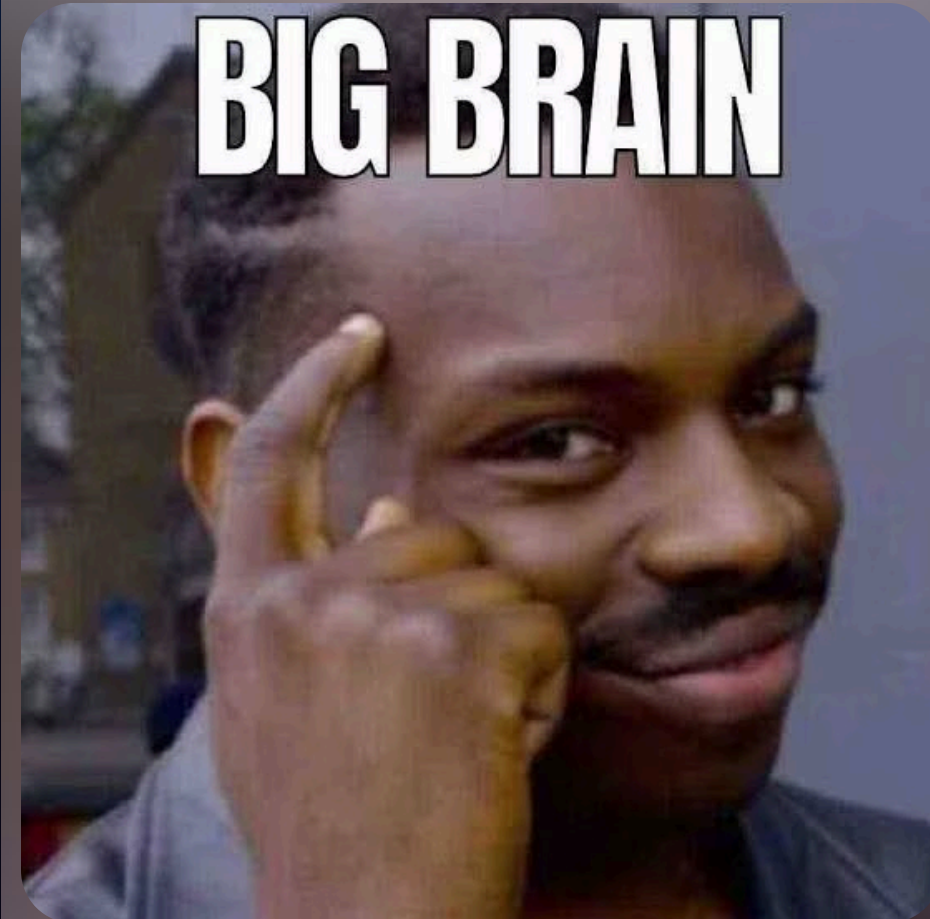
How can you solve it?

The boring way:

- You can solve a POMDP using either exact or approximate methods. **Exact algorithms** guarantee optimal policies but are computationally **heavy** and only practical for very **small state spaces**. To handle larger real-world problems, **approximate algorithms sacrifice absolute optimality** for massive computational speed-ups.

The funny way:

- The Big Brain Move, **imitational learning**. It's a **technique where an agent learns to perform a task by observing and mimicking human or expert demonstrations**. It is used when it is easier to show how to do something than to write complex, rule-based code or define exact reward functions.



Memory and Interpretability

- **RNNs**

- Standard approach: recurrent neural networks (e.g., **LSTMs**)
- Behavioral Cloning LSTMs learn an **implicit memory representation**. However, they are often **black-box models**
- difficult to interpret what is being remembered
- limited transparency in decision-making

- **Finite State Controllers**

- simpler and interpretable alternative
- explicit and structured memory

- **Project Goal**

- compare LSTMs with FSCs

Project Overview



Environment

Custom POMDP gridworld with 4 binary wall sensors providing partial observations.

Expert Policy

Recurrent PPO generates demonstration trajectories.

Maze Scales

3×3, 6×6, and 9×9 grids.

Evaluation

Success rate, reward, steps, inference cost, robustness, and interpretability.

Key Experiments

01

Recurrent PPO Training

Train expert agents per maze size; save best checkpoints.

03

FSC Training

Train FSC with varying internal memory states M .

05

Quantitative Comparison

Compare success rate, reward, episode length, and parameters.

02

Trajectory Collection

Build demonstration datasets from expert rollouts.

04

BC-LSTM Training

Train sequence model to imitate expert actions.

06

Robustness & Interpretability

Evaluate under action noise; visualize FSC internal states.

What is a RPPPO?

A **Recurrent PPO**, is a variant of the Proximal Policy Optimization algorithm that integrates Recurrent Neural Networks (RNNs), to handle **Partially Observable Markov Decision Processes (POMDPs)** where agents cannot fully observe the state space.

Why Use Recurrent PPO over the standard one?

Standard PPO assumes that the current observation gives the *complete* state of the environment. However, in many real-world scenarios this isn't true.

Stable Baseline3

```
# 1. Create the RecurrentPPO model (Use MlpLstmPolicy to include LSTM memory)
model = RecurrentPPO(
    policy="MlpLstmPolicy",
    env,
    learning_rate=lr,
    n_steps=256,
    seed=SEED,
)

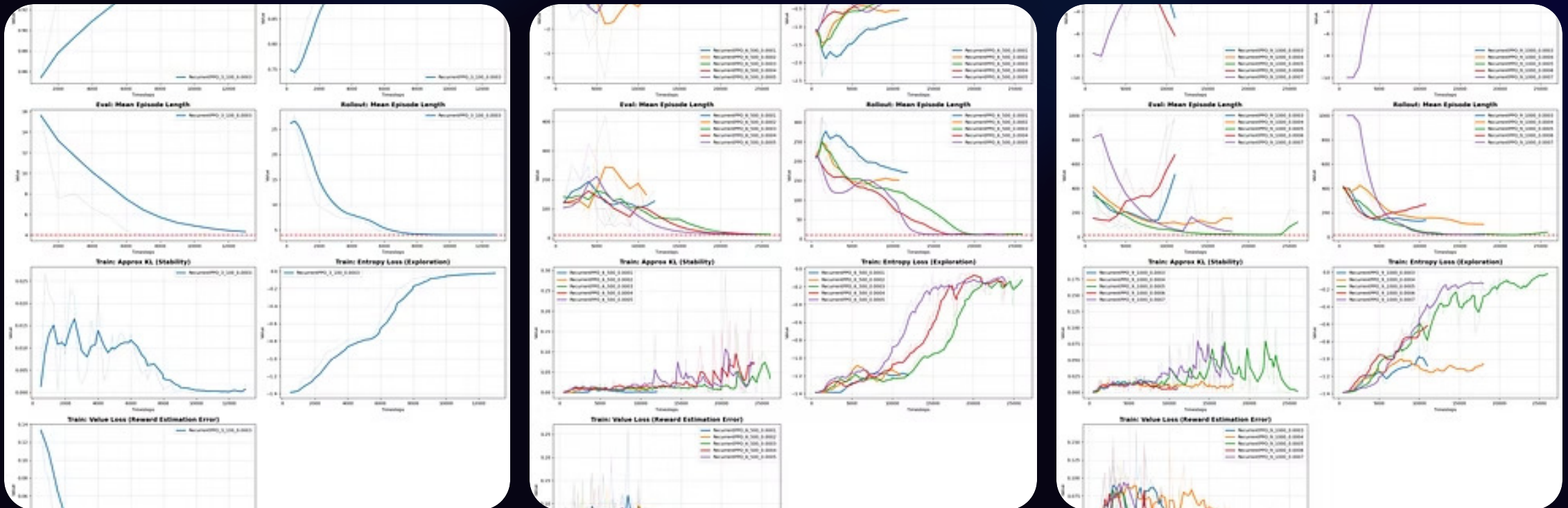
# 2. Set up the evaluation callback with early stopping based on model improvement
eval_callback = StopTrainingOnNoModelImprovement(
    eval_env,
    eval_freq=1000,
    max_no_improvement_evals=5, # Stop if the reward does not improve for 5 consecutive evaluations
    min_evals=5,              # Perform at least 5 evaluations before stopping
)

# 3. Train the model
model.learn(total_timesteps=total_timesteps, callback=eval_callback)
```

RPPO Training

For each maze size (3, 6, and 9), I aim to obtain an expert RPPO model. Since the algorithm is largely out-of-the-box, I only need to tune the learning rate and the maximum number of iterations allowed before truncating an episode.

The results show that RPPO performed remarkably well across all three maze sizes, consistently achieving optimal behavior by always finding the shortest possible path to the goal.



Trajectories Dataset

Problem: RPP0 is **too good**. The trajectories are **always** the **optimal** path to the goal.

"Shouldn't that be the result we are looking for?" Yes, but I want the agents' trajectories to include a bit of noise. This gives me two benefits:

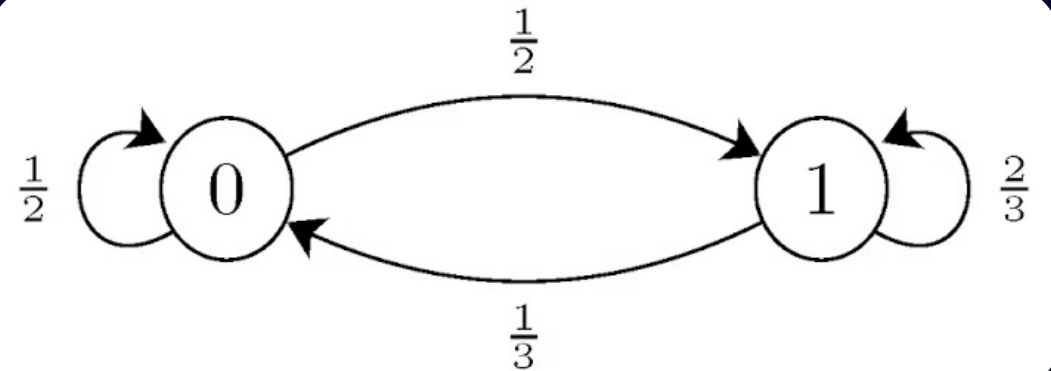
1. The agents will learn to move along trajectories that deviate slightly from the optimal one, which helps them recover and get back on track if something goes wrong.
2. It relaxes the assumption that the agent always behaves perfectly optimally. The agent may be subject to some bias or make small mistakes here and there.

```
Dataset for size=3x3:  
Number of trajectories: 200  
Average trajectory length: 5.33 steps  
Example trajectory (first 5 steps):  
Observations: [[1 1 0 1]  
               [0 0 1 1]  
               [0 0 1 1]  
               [1 0 0 0]  
               [0 0 1 1]]  
Actions: [2 2 1 2 1]  
Rewards: [-0.01 -0.01 -0.01 -0.01 1. ]
```

What is a FSC?

An FSC is a finite-state machine that controls the behavior of an agent by maintaining an internal discrete memory.

The brain of the FSC can be viewed as a cyclic graph, where each node represents a state, each edge represents a transition between states, and the belief represents the agent's internal estimate of the state it is currently in.



What does the FSC need?

It needs three integer dimensions:

- **M**: number of discrete internal memory state ($M = [1, 2, 3, 4, 5, \dots]$);
- **A**: number of possible actions ($[UP, DOWN, RIGHT, LEFT]$);
- **Y**: number of possible observations ($[0, 1]^4$).

The brain is made out by 3 components:

- **Initial state distribution**

$$\theta^{(\rho)} \in \mathbb{R}^M$$

- **Emission Policy**

$$\theta^{(\pi)} \in \mathbb{R}^{A \times M}$$

- **Internal transition model**

$$\theta^{(g)} \in \mathbb{R}^{M \times M \times A \times Y}$$

What is the objective of the FSC?

Given the **sequence of observations** y encountered by the expert while navigating the maze, the FSC should **generate the same sequence** of actions a with high probability.

Learning Objective: reproduce the expert demonstrations as accurately as possible.

Consequently, a likelihood function is required to measure how well the controller explains the collected trajectories.

Given a trajectory

$$T = (y_{0:T}, a_{0:T})$$

the probability assigned by the FSC is obtained by marginalizing over every possible sequence of internal memory states:

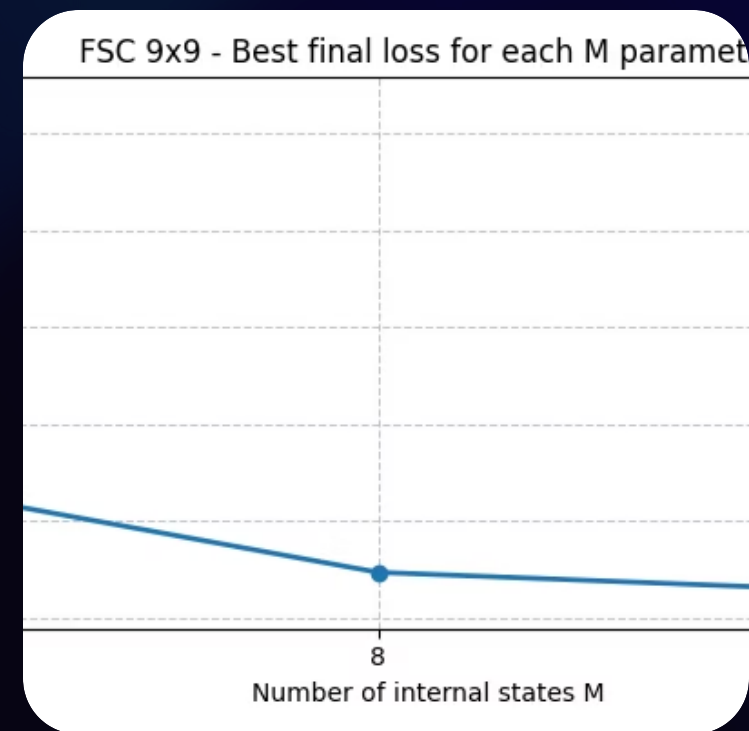
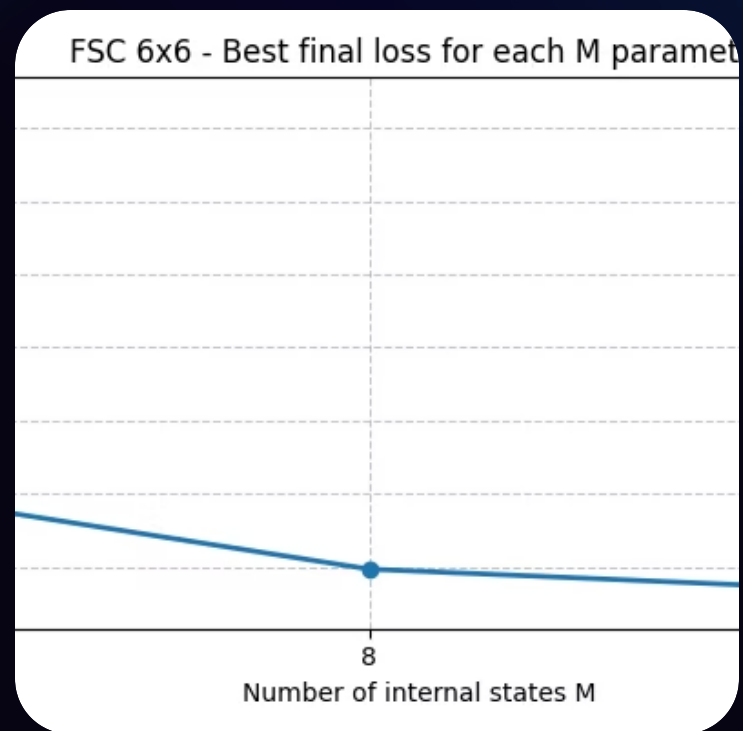
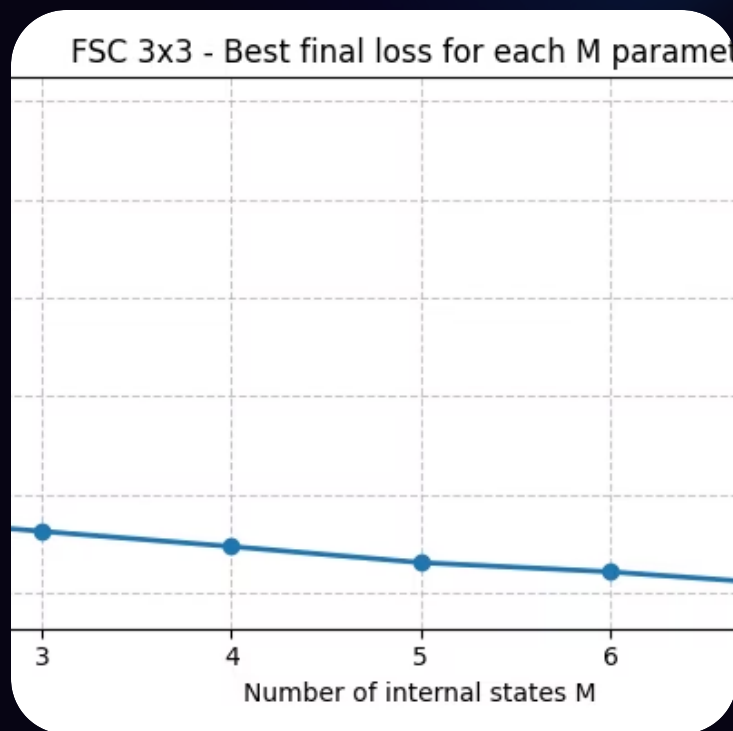
$$P(a_{0:T} \mid y_{0:T}) = \sum_{m_{0:T+1}} \rho(m_0) \prod_{t=0}^T \pi(a_t \mid m_t) g(m_{t+1} \mid m_t, a_t, y_t)$$

The model is trained by minimizing the average Negative Log-Likelihood (NLL) over all trajectories in the dataset

$$\mathcal{L} = -\frac{1}{N} \sum_{k=1}^N \log P \left(a_{0:T_k}^{(k)} \mid y_{0:T_k}^{(k)} \right).$$

FSC Training Results

FSC tested across $M \in \{1-9\}$ for 3x3 and $M \in \{1,2,4,8,16\}$ for larger mazes. $M = 4$ offered the best performance-efficiency trade-off.



What is a BC-LSTM?

The BC-LSTM is a recurrent neural network trained to **strictly mimic an expert's behavior**. By combining the current observation with its continuous hidden memory, the network learns an implicit representation of the environment to predict the expert's next action.

The fundamental difference between Behavioral Cloning and the FSC lies in this internal memory: while the BC-LSTM relies on a "black-box" hidden state, the FSC explicitly models a transparent, interpretable set of discrete states and probabilistic transitions.

PyTorch

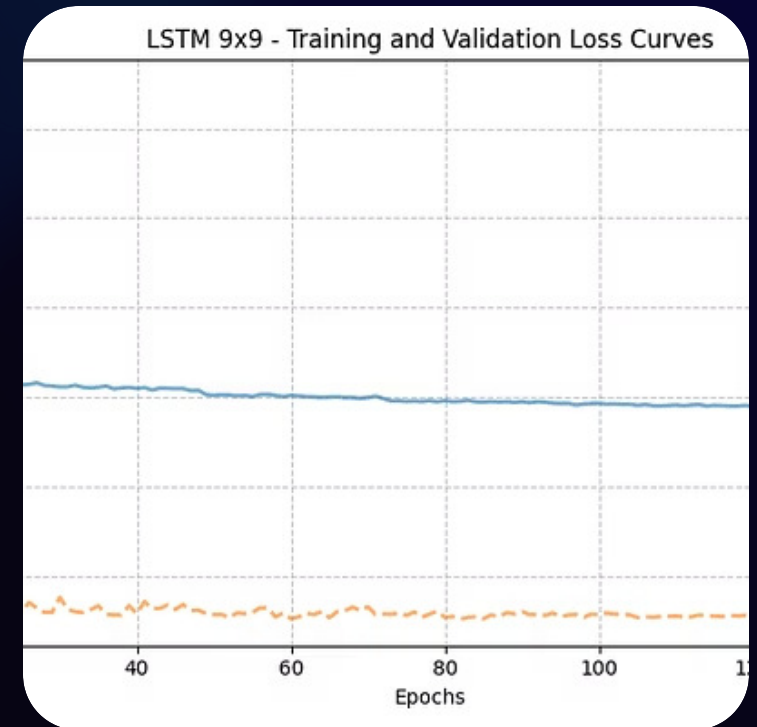
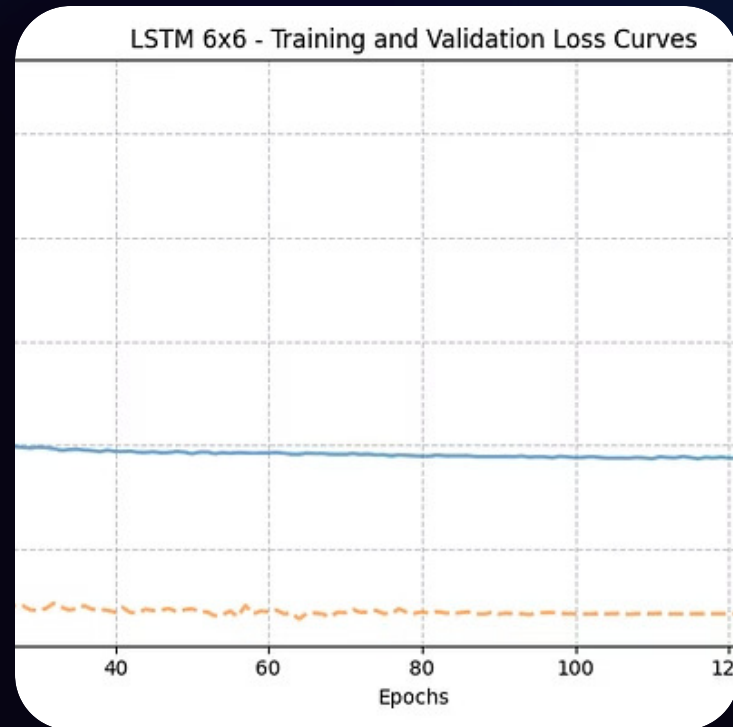
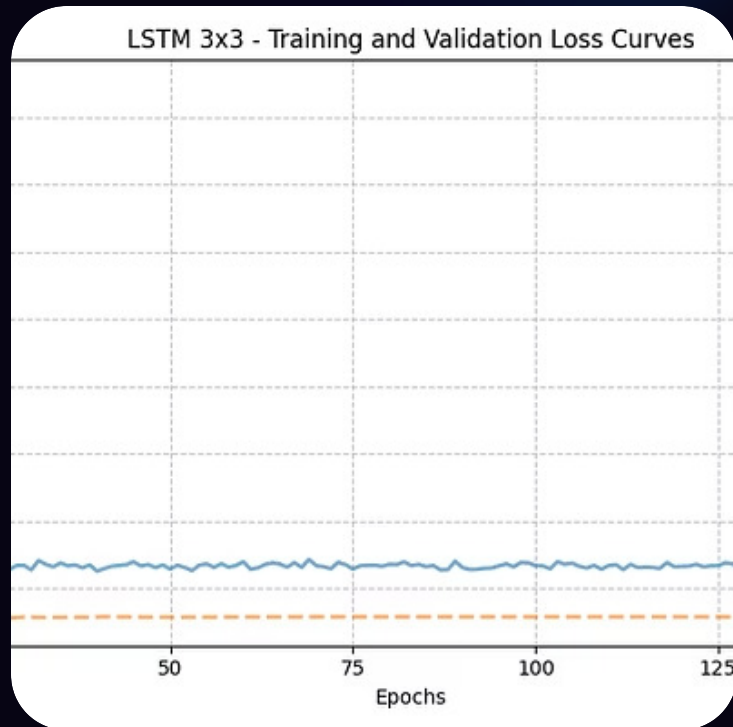
```
self.hidden_size = 32
self.num_layers = 1

self.encoder = nn.Sequential(
    nn.Linear(observation_dim, hidden_size),
    nn.ReLU(),
    nn.Linear(hidden_size, hidden_size),
    nn.ReLU()
)

self.lstm = nn.LSTM(hidden_size,
                    hidden_size,
                    num_layers=num_layers,
                    batch_first=True)
self.dropout = nn.Dropout(0.2)
self.head = nn.Linear(hidden_size, n_actions=4)
self.layer_norm = nn.LayerNorm(hidden_size)
```

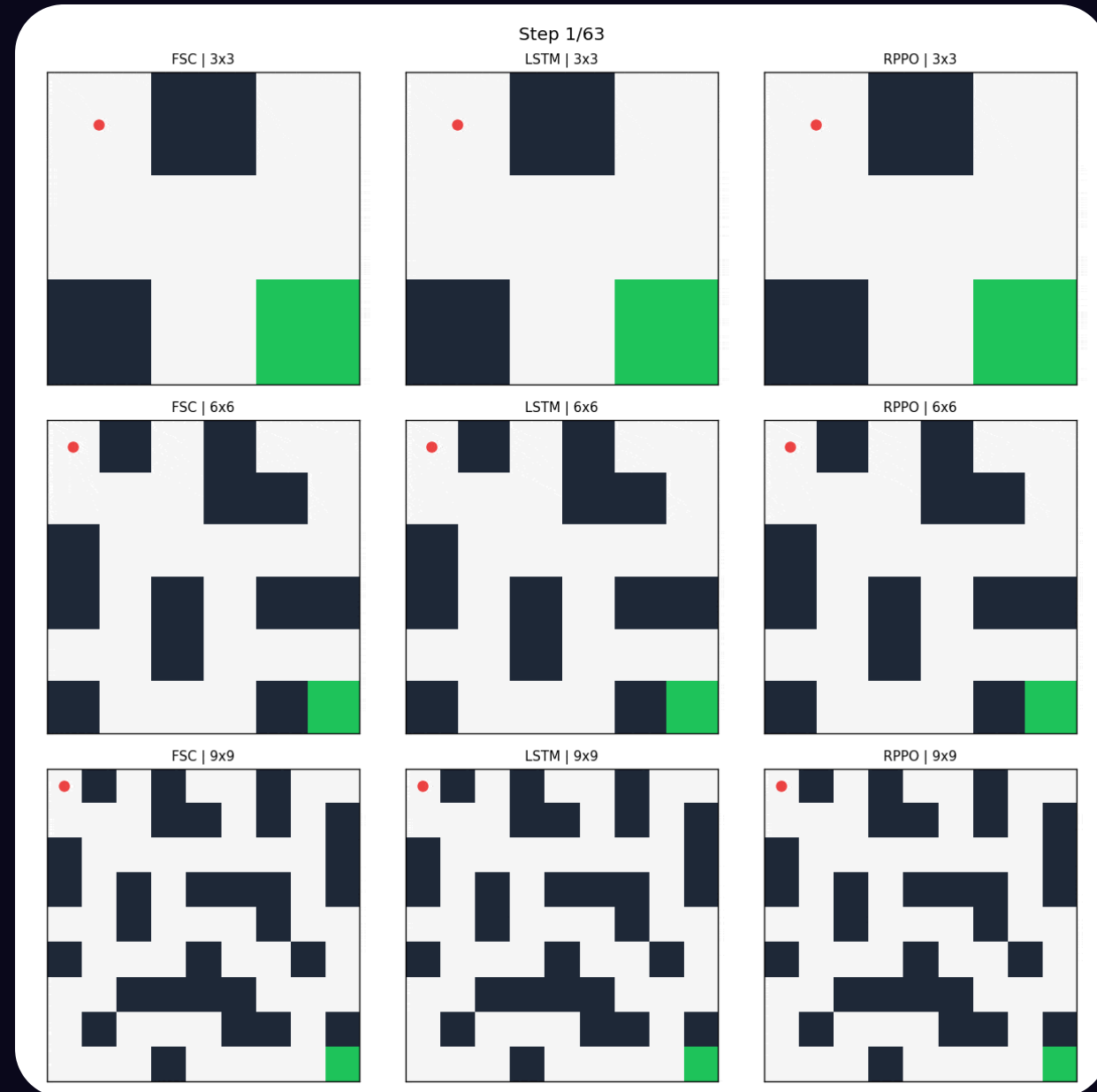
LSTM Training Results

LSTM hidden sizes scaled with maze complexity: 32 (3×3), 64 (6×6), 128 (9×9). One recurrent layer for 3×3 ; two layers for larger mazes.



Trajectory Comparison

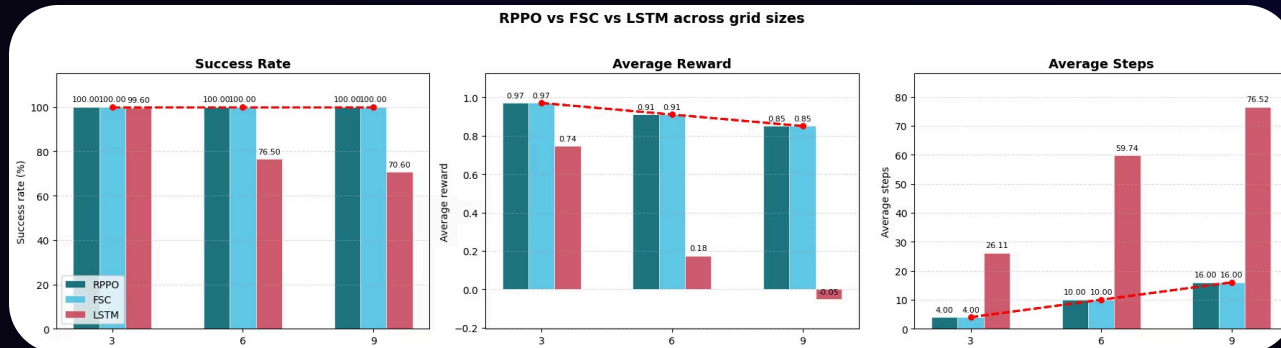
RPPO and FSC follow **optimal trajectories** across all maze sizes. The LSTM struggles on 6×6 and 9×9, where longer temporal dependencies are required for reliable imitation.



Performance & Parameter Efficiency

Success Rate & Reward

RPPO and FSC achieve very similar performance. LSTM is less stable — longer episodes, lower reward, and noticeably lower success rate.



Parameter Count

Maze	FSC	LSTM
3×3	1,044	9,860 (9.4x)
6×6	1,044	71,428 (68x)
9×9	1,044	282,116 (270x)

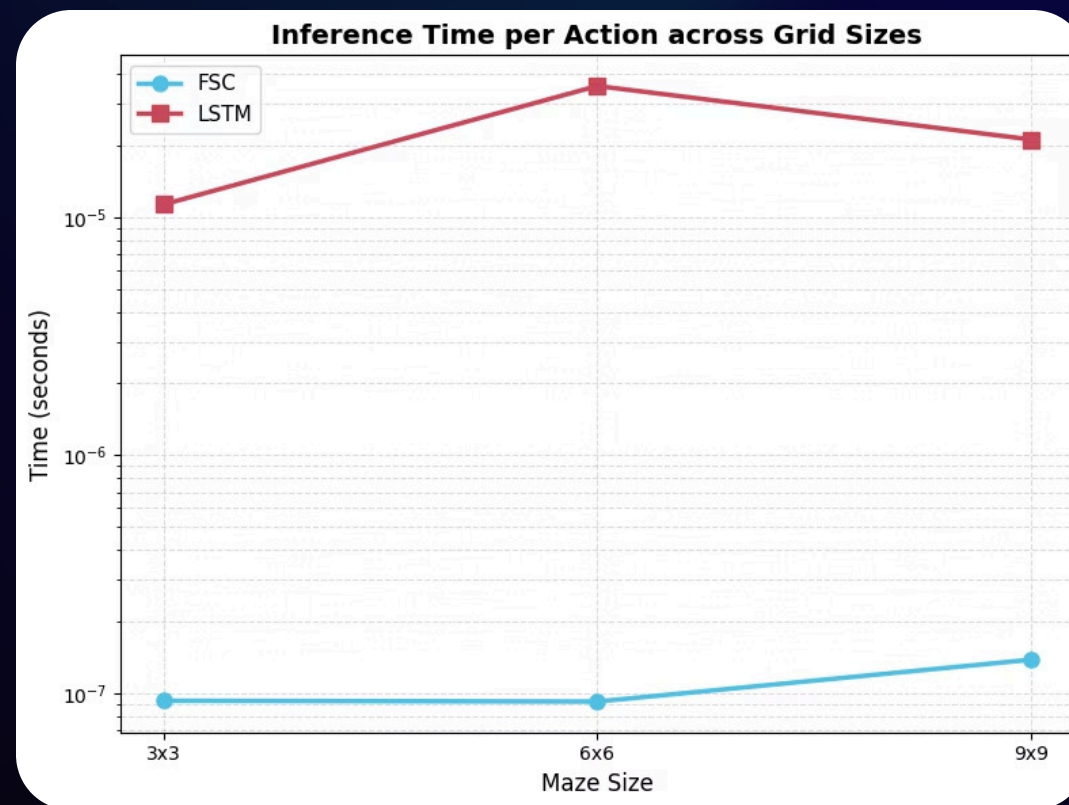
FSC stays constant (M=4, A=4, Y=16)
across all sizes.

Inference Time

Inference Speed

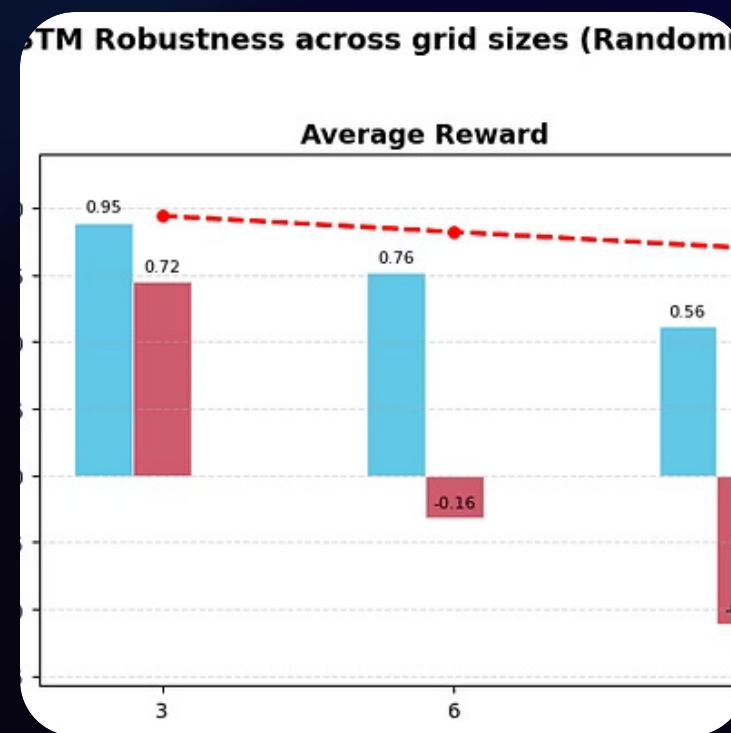
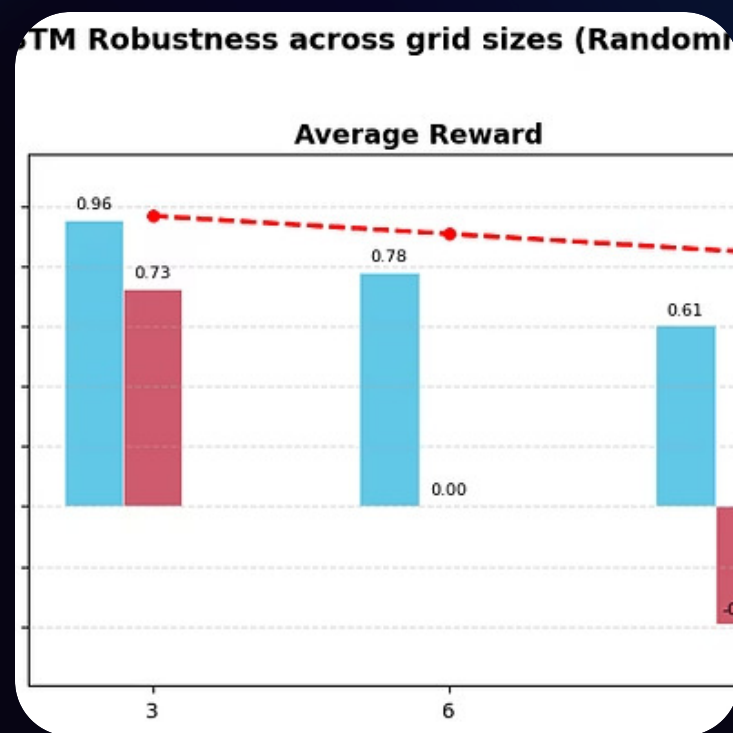
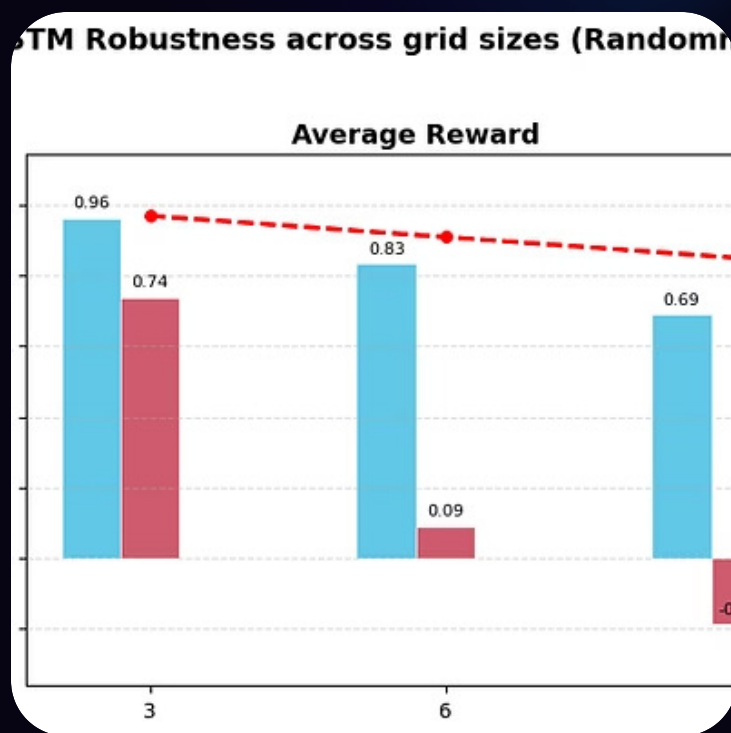
- FSC: $\sim 10^{-7}$ s per step.
- LSTM: $\sim 10^{-5}$ s.

A 100× gap consistent with parameter count. LSTM trains faster on GPU, but FSC inference is dramatically cheaper.

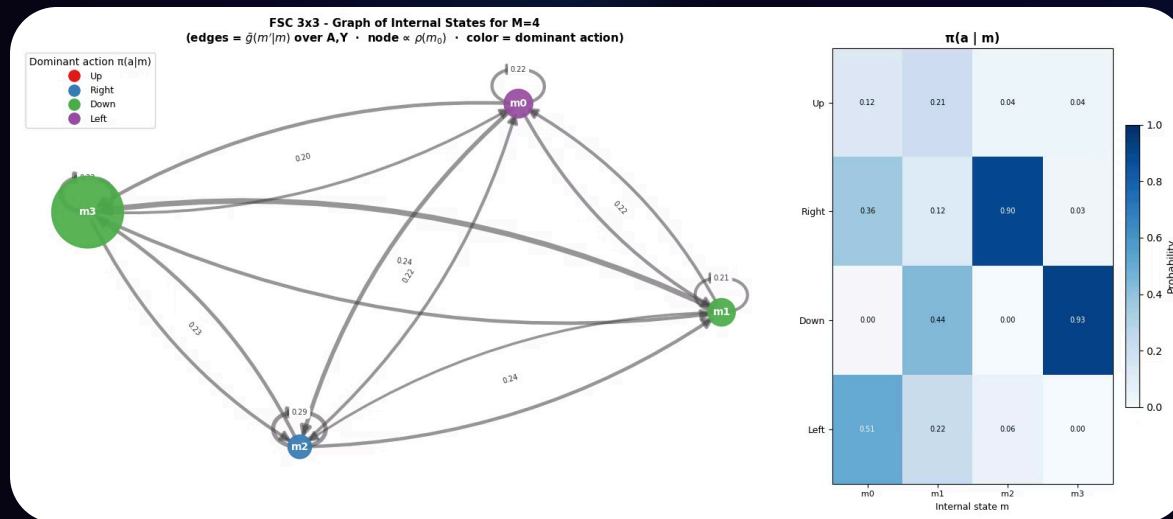


Robustness to Noise

At randomness levels 0.1-0.3, FSC remains stable and often recovers. LSTM degrades quickly once pushed off its nominal trajectory, accumulating extra steps and lower reward.



Interpretability



FSC: Transparent Reasoning

With just 4 internal states on the 3x3 maze, the FSC's policy graph is fully inspectable. Step-by-step belief updates reveal why each action is chosen.

LSTM: Opaque Approximator

A generic recurrent function approximator minimizing imitation loss — no direct, readable explanation of decisions.

i Interpretability is where the difference between the two approaches becomes most evident.

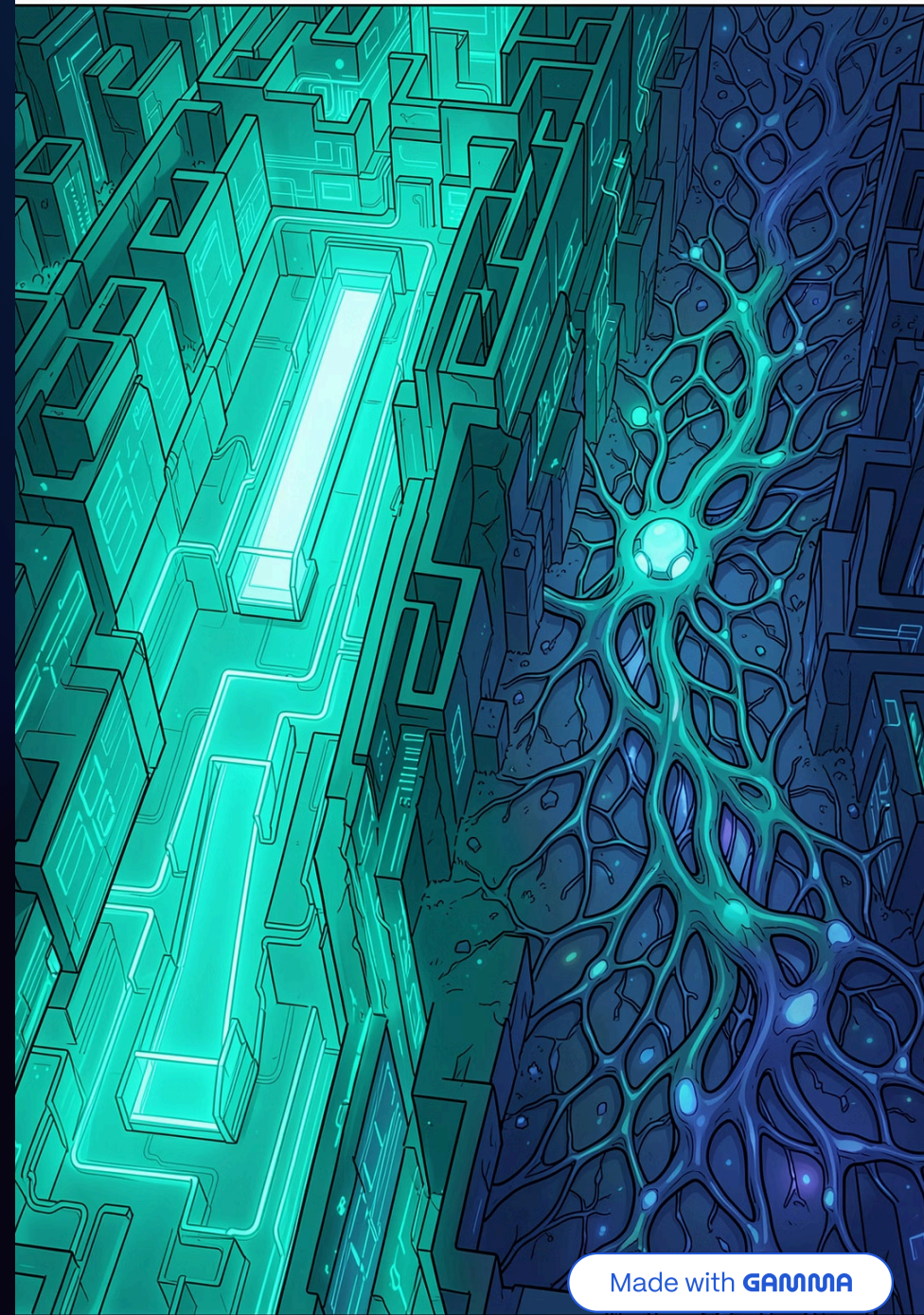
Conclusion

FSC — Recommended

Matches the expert, outperforms LSTM on larger mazes, uses **far fewer parameters**, lower inference cost, robust under noise, and fully interpretable.

LSTM — Practical Baseline

Flexible, fast to train with GPU, but harder to interpret, less parameter-efficient, and more sensitive to execution noise and long-horizon memory demands.



Thank you for your attention!!

Sources:

- [Automatic Generation and Learning of Finite-State Controllers](#)
- [Finite-State Controllers for \(Hidden-Model\) POMDPs using Deep Reinforcement Learning](#)
- [Sparse Stochastic Finite-State Controllers for POMDPs](#)
- [Learning finite-state controllers for partially observable environments](#)
- [Imitation learning of stabilizing policies for nonlinear systems](#)
- [Implicit Behavioral Cloning](#)
- [Developing Path Planning with Behavioral Cloning and Proximal Policy Optimization for Path-Tracking and Static Obstacle Nudging](#)
- [Behavioral Cloning in Reinforcement Learning](#)
- [Hands-On Imitation Learning: From Behavior Cloning to Multi-Modal Imitation Learning](#)